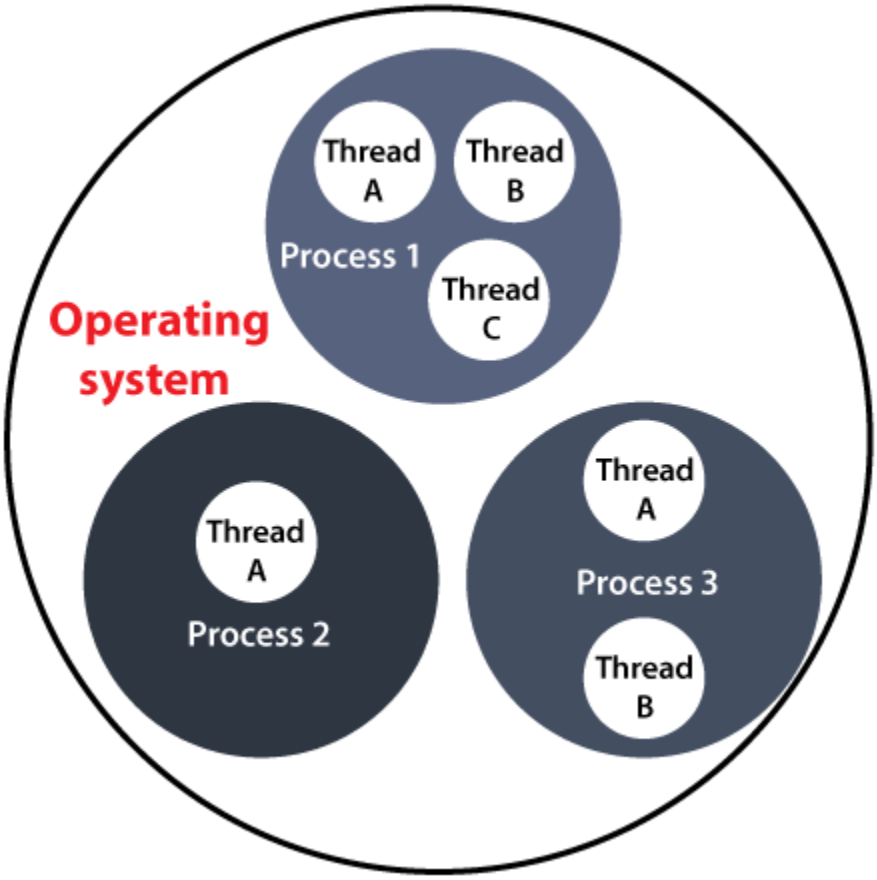
Thread Concept in Java

Before introducing the **thread concept**, we were unable to run more than one task in parallel. It was a drawback, and to remove that drawback, **Thread Concept** was introduced.

A **Thread** is a very light-weighted process, or we can say the smallest part of the process that allows a program to operate more efficiently by running multiple tasks simultaneously.

In order to perform complicated tasks in the background, we used the **Thread concept in Java**. All the tasks are executed without affecting the main program. In a program or process, all the threads have their own separate path for execution, so each thread of a process is independent.



Another benefit of using **thread** is that if a thread gets an exception or an error at the time of its execution, it doesn't affect the execution of the other threads. All the threads share a common memory and have their own stack, local variables

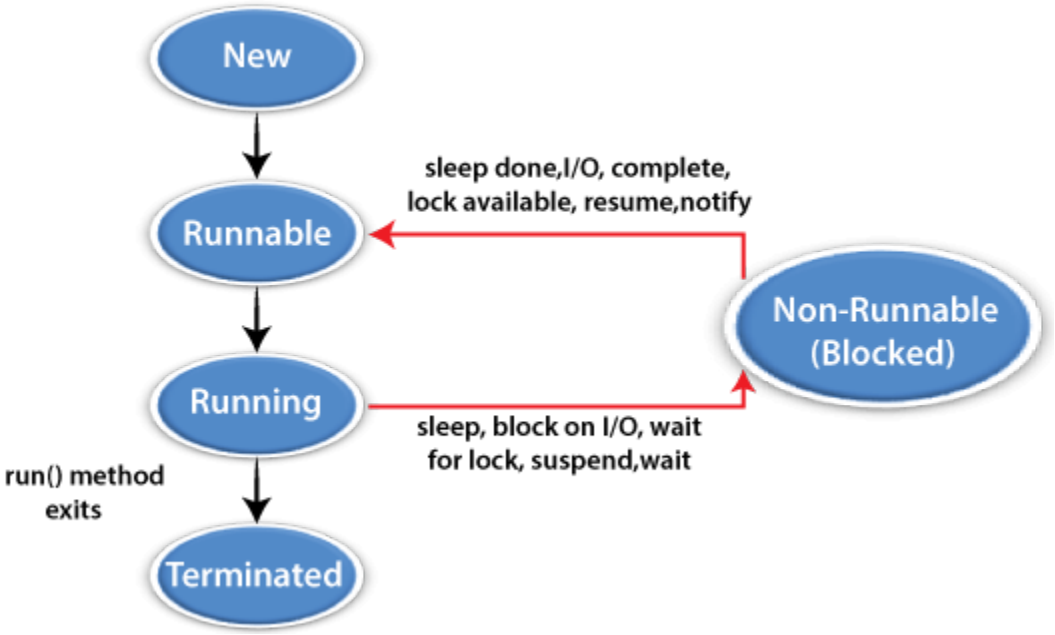
and program counter. When multiple threads are executed in parallel at the same time, this process is known as **Multithreading**.

In a simple way, a Thread is a:

- Feature through which we can perform multiple activities within a single process.
- Lightweight process.
- Series of executed statements.
- Nested sequence of method calls.

Thread Model

Just like a process, a thread exists in several states. These states are as follows:



1) New (Ready to run)

A thread is in **New** when it gets CPU time.

2) Running

A thread is in **a Running** state when it is under execution.

3) Suspended

A thread is in the **Suspended** state when it is temporarily inactive or under execution.

4) Blocked

A thread is in the **Blocked** state when it is waiting for resources.

5) Terminated

A thread comes in terminate state when at any given time, it halts its execution immediately.

Creating Thread

A thread is created either by "creating or implementing" the **Runnable Interface** or by extending the **Thread class**. These are the only two ways through which we can create a thread.

Let's dive into details of both these way of creating a thread:

Thread Class

A **Thread class** has several methods and constructors which allow us to perform various operations on a thread. The Thread class extends the **Object** class. The **Object** class implements the **Runnable** interface. The thread class has the following constructors that are used to perform various operations.

- **Thread()**
- **Thread(Runnable, String name)**
- **Thread(Runnable target)**
- **Thread(ThreadGroup group, Runnable target, String name)**
- **Thread(ThreadGroup group, Runnable target)**
- **Thread(ThreadGroup group, String name)**
- **Thread(ThreadGroup group, Runnable target, String name, long stackSize)**

Runnable Interface(run() Method)

The Runnable interface is required to be implemented by that class whose instances are intended to be executed by a thread. The runnable interface gives us the **run()** method to perform an action for the thread.

start() Method

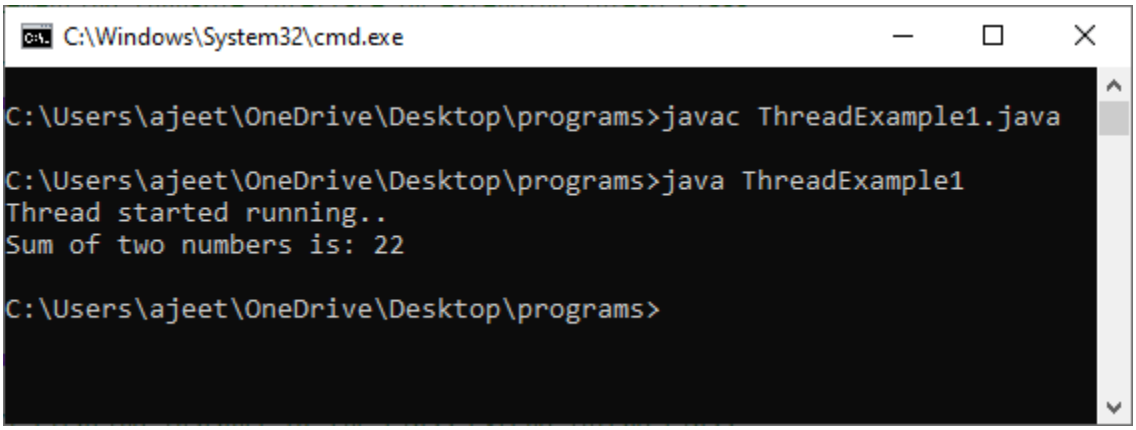
The method is used for starting a thread that we have newly created. It starts a new thread with a new call stack. After executing the **start()** method, the thread changes the state from New to Runnable. It executes the **run() method** when the thread gets the correct time to execute it.

Let's take an example to understand how we can create a Java thread by extending the Thread class:

ThreadExample1.java

```
// Implementing runnable interface by extending Thread class
public class ThreadExample1 extends Thread {
    // run() method to perform action for thread.
    public void run()
    {
      int a= 10;
      int b=12;
      int result = a+b;
      System.out.println("Thread started running..");
      System.out.println("Sum of two numbers is: "+ result);
    }
    public static void main( String args[] )
    {
     // Creating instance of the class extend Thread class
      ThreadExample1 t1 = new  ThreadExample1();
      //calling start method to execute the run() method of the Thread class
      t1.start();
    }
}
```

Output:



Creating thread by implementing the runnable interface

In Java, we can also create a thread by implementing the runnable interface. The runnable interface provides us both the run() method and the start() method.

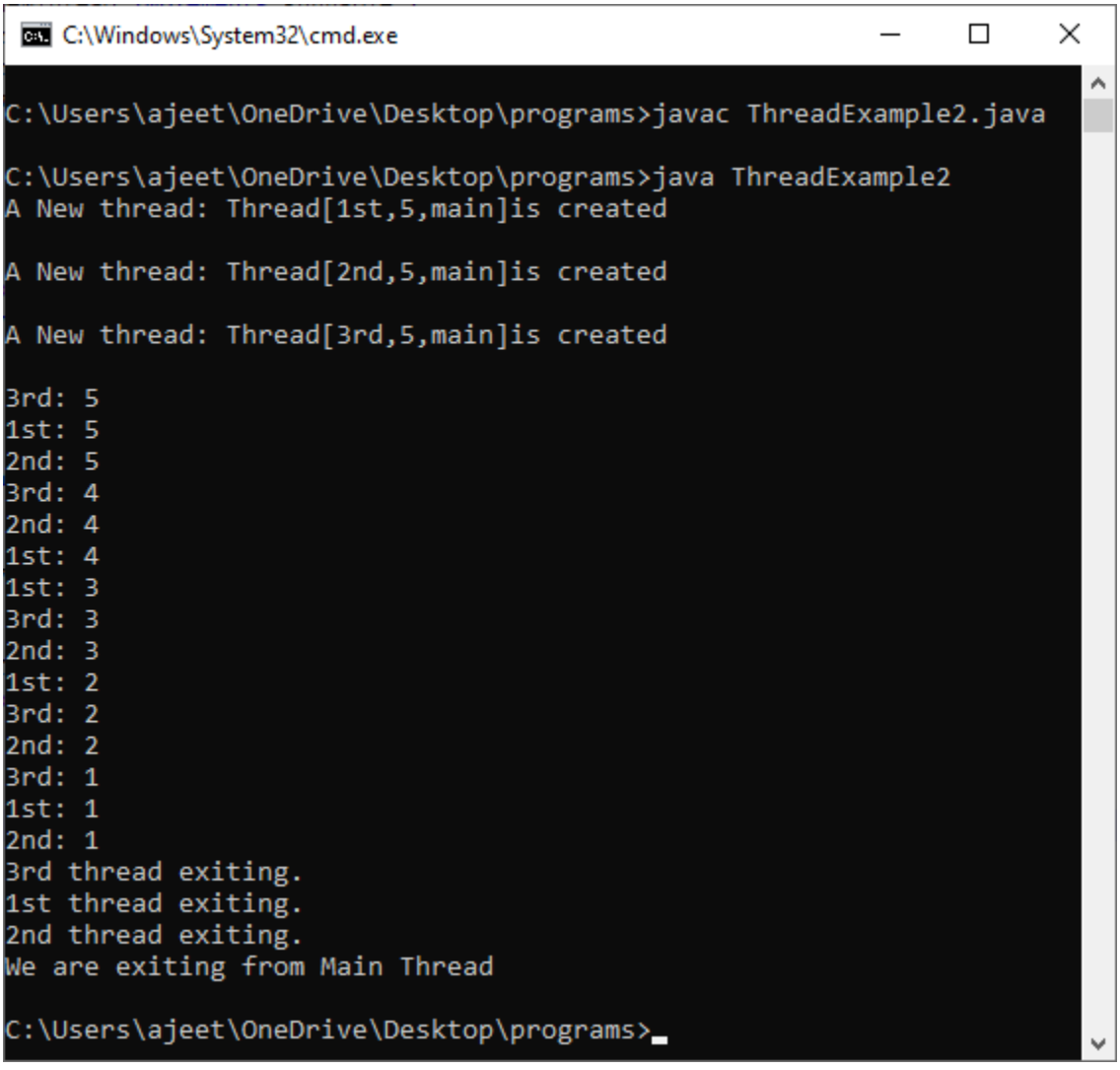
Let's takes an example to understand how we can create, start and run the thread using the runnable interface.

ThreadExample2.java

```
class NewThread implements Runnable {
   String name;
   Thread thread;
   NewThread (String name){
      this.name = name;
      thread = new Thread(this, name);
      System.out.println( "A New thread: " + thread+ "is created\n" );
      thread.start();
   }
   public void run() {
   try {
      for(int j = 5; j > 0; j--) {
         System.out.println(name + ": " + j);
         Thread.sleep(1000);
      }
   }catch (InterruptedException e) {
      System.out.println(name + " thread Interrupted");
   }
    System.out.println(name + " thread exiting.");
   }
```

```
}
class ThreadExample2 {
   public static void main(String args[]) {
      new NewThread("1st");
      new NewThread("2nd");
      new NewThread("3rd");
      try {
         Thread.sleep(8000);
      } catch (InterruptedException excetion) {
         System.out.println("Inturruption occurs in Main Thread");
      }
      System.out.println("We are exiting from Main Thread");
   }
}
```

Output:



In the above example, we perform the Multithreading by implementing the runnable interface.

Thread Synchronization

In multithreaded environments, it's essential to synchronize access to shared resources to prevent data corruption and ensure consistency. Java provides synchronization mechanisms such as the synchronized keyword and the java.util.concurrent package to facilitate thread synchronization.

The synchronized keyword can be used to create synchronized methods or blocks

```
class Counter
{
   private int count = 0;
   public synchronized void increment()
{
      count++;
   }
}
```

Alternatively, the java.util.concurrent package provides higher-level concurrency utilities, including locks, semaphores, and atomic variables, to manage thread synchronization more effectively.

Thread Pools

Creating and managing threads individually can be inefficient, especially in applications with a large number of tasks. Thread pools provide a solution by reusing existing threads to execute multiple tasks, thereby reducing the overhead associated with thread creation and destruction.

Java's ExecutorService and ThreadPoolExecutor classes facilitate the creation and management of thread pools:

```
ExecutorService executor = Executors.newFixedThreadPool(5);
executor.execute(new MyRunnable());
```

Thread Priorities

Java allows developers to assign priorities to threads, influencing the scheduling decisions made by the JVM's thread scheduler. Threads with higher priorities are given preference by the scheduler, but thread priorities are only hints and not strict guarantees of execution order.

```
Thread thread1 = new Thread();
thread1.setPriority(Thread.MAX_PRIORITY); // Highest priority
Thread thread2 = new Thread();
thread2.setPriority(Thread.MIN_PRIORITY); // Lowest priority
```

Inter-thread Communication

Threads often need to communicate with each other to coordinate their activities or exchange data. Java provides mechanisms such as wait(), notify(), and notifyAll() methods, along with the synchronized keyword, to facilitate inter-thread communication and synchronization.

Daemon Threads

Daemon threads are low-priority threads that run in the background, providing services to other threads or performing tasks such as garbage collection. Daemon threads automatically terminate when all non-daemon threads in the program have finished execution.

```
Thread daemonThread = new Thread();
daemonThread.setDaemon(true); // Set as daemon thread
```

Thread Safety

Ensuring thread safety is essential in multithreaded applications to prevent race conditions, data corruption, and other concurrency-related issues. Techniques such as using thread-safe data structures, employing synchronization, and minimizing shared mutable state help in achieving thread safety.

Thread Interruption

Java provides the interrupt() method to interrupt a thread's execution, signalling it to stop its current activities and terminate gracefully. Threads can check their interrupted status using the isInterrupted() method or catch InterruptedException to handle interruptions appropriately.

```
Thread thread = new Thread();
thread.interrupt(); // Interrupt the thread
```

Thread Local Variables

Thread-local variables are variables that have their own separate copy for each thread, ensuring that each thread accesses its own instance without interference from other threads. Java's **ThreadLocal** class provides a convenient way to create and manage thread-local variables.

```
ThreadLocal<Integer> threadLocal = ThreadLocal.withInitial(() -> 0);
threadLocal.set(42); // Set thread-local variable
int value = threadLocal.get(); // Get thread-local variable
```

Conclusion

In the world of Java programming, understanding the thread concept is paramount for building responsive, scalable, and efficient applications. Threads enable developers to leverage the full potential of modern multicore processors by executing multiple tasks concurrently.

By mastering thread creation, lifecycle management, synchronization, and thread pools, Java developers can unlock the power of multithreading and develop high-performance software solutions.

Java thread concept MCQ

1. Which method is used to start a newly created thread?

1. run()
2. start()
3. init()
4. begin()

Show Answer Workspace

---

2. Which interface must be implemented to create a thread by implementing the Runnable interface?

1. Thread
2. Runnable
3. Callable
4. Future

Show Answer Workspace

---

3. What is the default priority of a newly created thread in Java?

1. 1
2. 5
3. 10
4. 0

Show Answer Workspace

---

4. Which method is used to pause the execution of a thread for a specified time?

1. wait()
2. sleep()
3. join()
4. notify()

Show Answer Workspace

---

5. Which keyword is used to ensure that a block of code is executed by only one thread at a time?

1. synchronized
2. volatile
3. static
4. transient

Show Answer | Workspace

Next Topic Upcasting and Downcasting in Java

← prev | next →

Learn Important Tutorial

Python

Java

Javascript

HTML

Database

PHP

C++

React

B.Tech / MCA

DBMS

Data Structures

DAA

Operating System

Computer Network

Compiler Design

Computer Organization

Discrete Mathematics

Ethical Hacking

Computer Graphics

Web Technology

Software Engineering

Cyber Security

Automata

C Programming

C++

Java

.Net

Python

Programs

Control System

Data Warehouse

Preparation

Aptitude

Reasoning

Verbal Ability

Interview Questions

Company Questions